

4-Bit Signed ALU

Complete Project Guide - simulation, hardware build, testing, and Arduino display

This guide explains the whole ALU at project level. It is not a wire-by-wire manual, but it gives enough detail to understand how every module works, how the blocks connect, and how the final system is tested.

Item	Description
Project type	4-bit signed ALU using two's complement arithmetic
Input range	1000 to 0111, representing -8 to +7
Main operations	ADD, SUB, signed compare, signed multiply
Simulation	Logisim and TinkerCAD
Hardware	74HC/74LS logic ICs on breadboards
Display	Arduino Nano reads hardware outputs and shows selected result on LCD1602 I2C

1. Project Idea

The project is a 4-bit signed ALU built from real digital logic ICs. The inputs A and B are treated as two's complement numbers. The ALU can add, subtract, compare, and multiply signed values. The arithmetic itself is handled by hardware logic; the Arduino is only used as a display/readout layer.

A[3:0], B[3:0] -> ALU hardware -> operation outputs
Arduino reads outputs -> LCD display

The project was designed in modules so each section can be simulated, tested, then connected to the full build. This reduces debugging problems and makes the physical circuit easier to verify.

2. Top-Level Architecture

The ALU is divided into independent blocks. Some blocks operate directly on the original signed inputs. Other blocks convert signed inputs into magnitudes first and then apply signed correction at the end.

Block	Purpose
Input section	DIP switches provide A, B, SUB, and selection signals. Pulldowns keep inputs stable.
Absolute value / 2's complementer	Converts signed A and B into magnitudes for multiplication.
ADD/SUB unit	Performs $A + B$ or $A - B$ using XOR and a 4-bit adder.
Comparator	Uses an unsigned comparator first, then corrects result using sign bits.
Multiplier	Multiplies magnitudes using partial products and adders.
Signed product correction	Converts magnitude product into final signed 8-bit two's complement result.
Arduino display system	Reads hardware outputs using shift registers and displays selected result.

A, B inputs
|
+--> ADD/SUB -----> 4-bit sum + overflow
+--> Comparator -----> GT / EQ / LT
+--> Absolute value -> Multiplier -> Signed product correction -> 8-bit product

Arduino display reads the output lines and shows the selected operation result.

3. Input and Control Signals

The input section provides four bits for A and four bits for B. Each input is a digital line that must be either LOW or HIGH. Since floating inputs caused unstable behavior during the physical build, the final wiring uses strong pulldown resistors on the DIP switch lines.

Signal	Meaning
A3 A2 A1 A0	First signed 4-bit input
B3 B2 B1 B0	Second signed 4-bit input
A3, B3	Sign bits. 0 = positive/non-negative, 1 = negative
SUB	0 for addition, 1 for subtraction
SEL1 SEL0	Display selection lines for the Arduino/LCD readout

All switch inputs use pulldowns. For this build, 1k ohm pulldowns are preferred because they keep the lines more strongly at LOW on a large breadboard circuit.

4. ADD/SUB Unit

The ADD/SUB unit is based on one standard two's complement trick: subtraction can be performed by inverting B and adding 1. The SUB signal controls both the inversion and the carry-in.

```
B_eff = B XOR SUB
Result = A + B_eff + SUB
```

```
SUB = 0: Result = A + B
SUB = 1: Result = A + (~B) + 1 = A - B
```

The XOR gates conditionally invert B. The 74HC283 adder performs the actual addition. SUB is connected to the adder carry-in, with a local pulldown so the carry-in is definitely LOW when SUB = 0.

4.1 Overflow Flag

The overflow flag is used for signed two's complement arithmetic. It tells us that the 4-bit result wrapped outside the valid signed range. The range is -8 to +7, so examples such as 7 + 1 and -8 - 1 cannot be represented correctly in 4 bits.

```
Universal ADD/SUB overflow rule:
B_eff3 = B3 XOR SUB
Overflow = NOT(A3 XOR B_eff3) AND (A3 XOR S3)
```

This is different from Cout. Cout is useful for unsigned arithmetic and debugging, but Cout is not the signed overflow flag.

Example	4-bit result	Overflow
0111 + 0001	1000	1
1111 + 1111	1110	0
1000 - 0001	0111	1
0011 - 0001	0010	0

5. Absolute Value / Magnitude Getter

The multiplier is easier to build if it first receives positive magnitudes. Each signed input is converted to magnitude using conditional inversion and addition.

```
Magnitude = (Input XOR SignBit) + SignBit
```

If the sign bit is 0, XOR changes nothing and the adder adds 0. If the sign bit is 1, XOR inverts the bits and the adder adds 1, producing the two's complement magnitude.

Input	Meaning	Magnitude
0000	0	0000
0011	+3	0011
1111	-1	0001
1101	-3	0011
1000	-8	1000

6. Comparator

The comparator uses a 74HC85 to compare A and B as unsigned numbers first. The unsigned outputs are then corrected using the sign bits to become signed comparison outputs.

```
sameSign = NOT(A3 XOR B3)

GT = (NOT A3 AND B3) OR (sameSign AND U_GT)
EQ = U_EQ
LT = (A3 AND NOT B3) OR (sameSign AND U_LT)
```

If the signs are different, the positive number is greater. If the signs are the same, the unsigned comparator result is valid and can be used directly.

For the 74HC85 cascade inputs in standalone 4-bit mode, use $I_{GT} = 0$, $I_{EQ} = 1$, and $I_{LT} = 0$. This tells the comparator that there is no previous comparison stage and that the previous stage should be considered equal.

7. Multiplier

The multiplier is built in two stages. First, the magnitudes are multiplied as unsigned 4-bit numbers. Then the 8-bit product is corrected based on the original signs of A and B.

The unsigned magnitude multiplier uses 16 AND gates to generate partial products and three 4-bit adders to combine the shifted rows.

$$P_{mag} = A_{mag} \times B_{mag}$$

Part	Function
16 AND gates	Generate A_i AND B_j partial products
Adder stage 1	Adds first two shifted partial-product rows
Adder stage 2	Adds stage 1 result with the third row
Adder stage 3	Adds stage 2 result with the fourth row

In the unsigned multiplier stages, each adder carry-in is tied to GND. The carry-out of one stage becomes a normal high bit input of the next addition stage, not the next adder carry-in.

7.1 Signed Product Correction

After the magnitude product is created, the final sign is calculated from the original sign bits.

$$\begin{aligned} \text{ProductSign} &= A_3 \text{ XOR } B_3 \\ \text{FinalProduct} &= (P_{mag} \text{ XOR } \text{ProductSign}) + \text{ProductSign} \end{aligned}$$

Here, + means binary addition, not OR. If ProductSign is 0, the product stays unchanged. If ProductSign is 1, every product bit is inverted and 1 is added, producing the 8-bit two's complement negative result.

Pmag	ProductSign	FinalProduct
00000110	0	00000110
00000110	1	11111010
00000001	1	11111111
00001000	1	11111000
01000000	0	01000000

8. Arduino Display System

The Arduino display system is included as the readout layer of the finished ALU. It does not calculate the ALU results. The hardware ICs still perform the arithmetic and logic operations. The Arduino only reads the output lines and displays the selected result clearly on an LCD1602 I2C module.

Because the ALU has many output lines, 74HC165 parallel-in serial-out shift registers are used. They allow the Arduino Nano to read many signals using only a few pins.

Signal group	Read by Arduino
ADD/SUB result	S0-S3 and overflow/Cout if connected
Comparator result	GT, EQ, LT
Multiplier result	FinalProduct P0-P7
Control signals	SUB, SEL0, SEL1

74HC165 chain -> Arduino Nano -> LCD1602 I2C

The Arduino reads the hardware outputs and chooses what to display based on SEL1/SEL0.

The display can show binary output and signed interpretation. For multiplication, it can display the full 8-bit result. For comparison, it can show GT, EQ, or LT.

9. IC Count Summary

The exact count depends on how separately each block is built. The full version including signed multiplication and final product correction uses the following logic ICs.

IC	Quantity	Used for
74HC283	8	Add/sub, absolute value, multiplier additions, signed product correction
74HC86	7	XOR inversion, SUB logic, sign logic, signed product correction
74HC08	5	Multiplier partial products and comparator correction
74HC04	1	NOT gates for sign correction
74HC32	1	OR gates for comparator correction
74HC85	1	Unsigned comparator core

Total logic ICs: 23

Without final signed multiplier correction: 18

The Arduino readout adds the Arduino Nano, LCD1602 I2C module, and three 74HC165 shift registers if all planned output/control lines are read through the serial chain.

10. Power, Capacitors, and Stability

The physical build uses many ICs, so clean power matters. Each logic IC should have a 100nF decoupling capacitor between VCC and GND. These capacitors reduce fast voltage dips when ICs switch. Larger electrolytic capacitors across the breadboard rails help smooth slower dips for the whole board.

Capacitors do not create or regulate 5V. They only help stabilize the voltage already supplied by the power source. A stable 5V supply and common ground across all breadboards are still required.

All unused HC inputs must be tied to GND or +5V. Floating inputs can cause random outputs, false highs, blinking LEDs, and unstable behavior.

11. Testing Strategy

The ALU is tested module by module before full integration. Each block has simple binary tests that confirm the expected output before connecting it to the next block.

Module	Key tests
ADD/SUB	0+0, 1+1, 7+1 overflow, 0-1, -8-1 overflow
Absolute value	0000, 0011, 1111, 1101, 1000
Comparator	equal, positive greater than negative, negative less than positive, same-sign comparisons
Multiplier magnitude	0 x anything, 1 x anything, 3 x 3, 8 x 8, 15 x 15 unsigned
Signed multiplier	positive x positive, positive x negative, negative x positive, negative x negative
Arduino display	Verify that selected hardware result matches LCD output

When debugging physical hardware, measurements should be taken directly at IC pins. LED behavior is useful, but voltage measurements are more reliable.

12. Build Notes

The circuit is large enough that layout matters. The multiplier and signed correction stages should be kept separated from the add/sub and comparator area if possible. This makes debugging easier and reduces wiring confusion.

A practical breadboard split is: input and add/sub on one area, comparator on another, magnitude preparation on another, multiplier on separate boards, and Arduino display wiring kept on the side. Power and ground rails must be connected clearly between all boards.

Recommended order:

1. Test input switches and pulldowns.
2. Test ADD/SUB.
3. Test overflow.
4. Test absolute value blocks.
5. Test comparator.
6. Test unsigned multiplier.
7. Test signed product correction.
8. Connect Arduino readout.
9. Run final operation tests.

13. Final System Behavior

The finished system accepts two 4-bit signed inputs and provides arithmetic and comparison outputs. The hardware modules generate the real results. The Arduino display system reads those results and presents them in a readable format.

Operation	Output
ADD	4-bit sum plus overflow flag
SUB	4-bit difference plus overflow flag
COMPARE	GT, EQ, LT
MUL	8-bit signed two's complement product

This makes the project more than a single ALU diagram: it includes simulation, physical logic, testing, and a practical output interface.